Text-Conditioning Style Absorption for Krea 2 RAW

An Integrated, CPU-Feasible, Dataset-Free Method for Packaging Known Style Priors as LoRA

A synthesis of the closed-form text-pathway draft and the revised TCSA protocol

Integrated methodological paper · September 2026

Editorial note on this integration

This document merges two sequential drafts of the same research program. The first draft, Zero-Shot Style Extraction and Closed-Form LoRA Distillation from Text-Conditioning Pathways, relocated style extraction from the infeasible 12B velocity field to the text-injection maps and gave a correct rank-1 closed-form construction. The second draft, Text-Conditioning Style Absorption for Krea 2 RAW (TCSA), kept that relocation, named the method, promoted thin-SVD / least-squares absorption to the primary synthesizer, and added null controls, a soft-prompt baseline, and a more falsifiable evaluation protocol. Neither draft is discarded. The integrated paper uses TCSA as the method name and experimental spine; restores the algebraically correct rank-1 initializer from the first draft; treats SVD/LS as the default synthesis route; and records, in an appendix, exactly what changed between the two sources.

Abstract

Low-Rank Adaptation (LoRA) is the standard mechanism for attaching a reusable visual style to a text-to-image foundation model. Conventional style LoRAs are trained from captioned images and therefore inherit the cost of dataset curation and GPU optimization. A recent feasibility idea proposed a different route: compare a pretrained model's own responses to a content prompt and a content-plus-style prompt, treat the difference as a style direction, and distill that direction into a LoRA without external images. Placing that difference in the flow-matching velocity field of Krea 2 RAW, a dense 12-billion-parameter diffusion transformer, is conceptually attractive and computationally hopeless on CPU. A single forward pass of the DiT is already impractical; back-propagating through the same backbone is not a research loop.

This paper relocates the extraction point to the text-conditioning pathway. Krea 2 encodes language with Qwen3-VL-4B, aggregates a fixed stack of hidden states, and injects those features through a small set of maps — `txtmlp` / `txt_in` and `txtfusion` / `text_fusion` — before packed tokens enter the DiT blocks. Those maps can be read from disk without running the transformer. The proposed method, Text-Conditioning Style Absorption (TCSA), encodes matched prompt pairs with the official text-only stack, tests whether the resulting shift is stable, content-independent, and style-specific, and absorbs the accepted subspace into the injection maps by a two-stage low-rank construction: a correct analytic rank-1 initializer followed by thin-SVD / least-squares refinement. The resulting file is a portable LoRA, validated externally on a hosted Krea 2 Turbo endpoint with a four-condition protocol that asks whether the style word can be retired.

The claim is deliberately narrow. TCSA does not invent an unseen style, does not match the high-frequency fidelity of an image-trained LoRA, and does not assert that visual style is universally linear in language space. It tests whether a style the pretrained model already recognizes can be converted from an explicit text condition into a reusable default prior, without a local GPU and without a style-image dataset.

Keywords: Krea 2, LoRA, flow matching, text conditioning, style transfer, low-rank regression, representation engineering, CPU inference

1. Introduction

Modern text-to-image models store a large inventory of visual styles in their pretraining. A user who writes a watercolor tiger walking through a jungle is not teaching the model watercolor; the model already associates that word with a cluster of rendering behaviors. Conventional LoRA training ignores this fact. It collects images, captions them, and optimizes a low-rank update until the model reproduces the dataset. The pipeline works, but it is expensive, and it leaves open whether the adapter captured a general style concept or the statistics of a particular folder of pictures.

The original feasibility study that motivates this work inverted the supervision. Instead of images teaching the model, the model would teach an adapter. For matched latents and timesteps one would compute a velocity difference

$$D_s = v_\theta(x_t, t, P_{c+s}) - v_\theta(x_t, t, P_c)$$

between a content-plus-style prompt and a content-only prompt.

If D_s contained a content-independent component, that component could be distilled into $\Delta W \approx BA$. The scientific question remains interesting. The engineering plan does not survive contact with a CPU-only machine. Krea 2 RAW is a dense 12B DiT. Collecting velocity tensors across concepts, seeds, and timesteps already implies on the order of 10^2–10^3 full transformer forwards. Fitting LoRA against those tensors requires student and teacher forwards plus backward passes through the same frozen backbone. Avoiding VAE decoding removes only a minor cost.

This paper keeps the original research question and changes the locus of extraction. Style-conditioned behavior is still treated as a difference between paired prompts. The difference is measured in the text-conditioning pathway rather than in the image velocity field. The adapter is constructed by a closed-form low-rank update of the linear maps that send text features into the DiT, not by gradient descent on the DiT itself. Visual evaluation remains external. The weaker and testable claim is not that style first appears in text conditioning, but that a style-dependent signal is already observable there when the pretrained model recognizes the style phrase.

1.1 Research question

Can a reusable Krea 2 style LoRA be constructed by comparing official text-conditioning responses to matched content and content-plus-style prompts, absorbing the resulting style subspace into low-rank updates of the text-injection layers, and performing the entire extraction on CPU, with image generation deferred to a hosted endpoint?

The question splits into five operational subquestions.

RQ1. Does the official Krea 2 text-conditioning stack produce a stable, cross-content shift when a known style phrase is added to a prompt?

RQ2. Can that shift be separated from residual semantic effects of the extra tokens, and from null controls that are not style renderers?

RQ3. Can the accepted shift be absorbed into low-rank updates of `txt_in` / `txtmlp` and `text_fusion` / `txtfusion` without executing the DiT?

RQ4. Does the resulting adapter move style-free prompts toward the explicit style condition on held-out contents?

RQ5. Can the complete extraction and packaging pipeline run within CPU RAM and disk constraints?

1.2 Contributions

This integrated paper makes five methodological contributions. First, it diagnoses why velocity-field self-distillation is not a CPU research method on a 12B flow-matching DiT, and it corrects an inconsistency in the original distillation objective. Second, it proposes a text-conditioning reformulation whose only local model is the official Qwen3-VL text encoder in text-only mode, plus a handful of linear weight matrices lazily read from the RAW checkpoint. Third, it specifies a two-stage synthesizer: an algebraically correct rank-1 initializer, then thin-SVD / least-squares absorption as the primary method. Fourth, it adds cheap admissibility tests, null controls, and a soft-prompt baseline that can reject an unstable or semantically contaminated direction before any LoRA file is written. Fifth, it specifies a four-condition external evaluation protocol that separates default-style transfer from simply reproducing an explicit style prompt.

2. Background and Related Work

2.1 LoRA for text-to-image models

LoRA parameterizes a weight update as ΔW = BA, with B $\in$ $R^{d\times r}$ and A $\in$ $R^{r\times k}$, r $\leq$ min(d, k) [1]. For diffusion and flow-matching transformers, the usual targets are attention projections and often MLP and conditioning maps. Official Krea 2 guidance recommends training LoRAs on the undistilled RAW checkpoint and applying them at inference on the 8-step Turbo checkpoint [3, 4, 5]. Default training recipes wrap all 264 Linear layers of the DiT at rank and alpha 32 [6]. That configuration is appropriate for image-supervised style and character adapters. It is the wrong default for a CPU extraction method, because it assumes the DiT will be present in the training loop.

2.2 Model-as-teacher style adapters

Concept Sliders demonstrated that a frozen diffusion model can supervise a LoRA from paired text conditions, without a large image collection, by aligning denoised scores under a target concept with scores under an attribute-augmented concept [2]. The present method belongs to the same family: the pretrained model is both source of knowledge and teacher. It differs in three respects. It targets a flow-matching DiT rather than an epsilon-prediction UNet. It refuses to back-propagate through the image backbone. And it treats style as a shift to be absorbed in the text-injection maps, which is a weaker but computationally local surrogate of score-space alignment. TCSA should therefore be read as representation engineering, not as a replacement for full velocity- or score-space distillation.

2.3 Krea 2 conditioning path

Krea 2 is a dense 12B single-stream DiT [3, 4]. Language is encoded by Qwen3-VL-4B-Instruct [8]. At inference the encoder is wrapped in a fixed instruction template and a stack of hidden states is taken from twelve layers (indices 2, 5, 8, …, 35), producing a tensor of shape (B, 12, S, 2560). That stack is consumed by a TextFusion component and then by a small text MLP before text and image tokens are packed into the DiT stream. In public loaders the injection maps appear under two namespaces: native / Musubi names such as `txtmlp.1`, `txtmlp.3`, `txtfusion.projector`; and Diffusers / PEFT names such as `txt_in.linear_1`, `txt_in.linear_2`, `text_fusion.projector`.

Community tooling independently treats this path as the site of reference addressing and style transfer. Layer-grouped reference LoRA loaders apply separate strengths to `text_fusion` / `txt_in` and to the DiT body, on the empirical claim that style and anti-bleed live in the conditioning maps. Utilities that strip DiT-block tensors from trained style LoRAs and keep only `txtfusion` often retain a large fraction of the style effect while shrinking the file. That split is the architectural reason a text-only extraction can hope to produce a visible style prior at all. It is also why exact key mapping and exact multi-layer aggregation are implementation-critical: a file that loads but targets the wrong matrices is an especially dangerous failure mode, because it can look valid and do nothing.

2.4 Why the original velocity pipeline fails on CPU

Let N_c be the number of semantic concepts, N_z the number of seeds, N_t the number of timesteps, and two prompts per pair. A modest grid of 10–20 concepts, 2–5 seeds and 4–8 timesteps already implies hundreds to thousands of DiT forwards before distillation begins. Each forward materializes a velocity tensor at image resolution in latent space. Distillation then repeats a student forward and a teacher forward at every optimizer step. On a CPU this is not slow research; it is a stalled pipeline. The redesign therefore treats DiT execution as an external validation service, not as a local primitive.

3. Problem Formulation

Let P_c be a content prompt and P_{c+s} the same content with a style phrase inserted. Let E(·) denote the official Krea 2 text-conditioning encoder, including the multi-layer aggregation used at inference. Let $Proj_W$ denote the composition of the selected text-injection linear maps with weights W drawn from RAW. Two issues in the motivating velocity-space draft must be repaired before that identity can even be discussed.

First, a draft objective that set $F_{\theta+\Delta\theta}(x_t, t, P) \approx D_s$ is dimensionally and semantically inconsistent. D_s is a difference of velocities, not a velocity. The internally consistent teacher–student target, if one were willing to run the DiT, would be

$$F_{\theta+BA}(x_t, t, P_c) \approx F_{\theta}(x_t, t, P_{c+s})$$

The adapter on a content-only prompt should reproduce the base model on the styled prompt.

Second, even the repaired objective is inaccessible without a GPU. The feasible surrogate is therefore restricted to the maps that consume E(P):

$$Proj_{W+BA}(\,E(P_c)\,) \approx Proj_{W}(\,E(P_{c+s})\,)$$

Hold the encoder and the DiT fixed. Edit only the text-injection linears.

A solution of this surrogate cannot be expected to reconstruct brush-level style. It can be expected, if RQ1 holds, to push the model's default conditioning toward the same region of text-feature space that the style phrase already occupies. The surrogate does not prove that image-space velocity will match. It only asks whether the conditioning representation can be transformed so that the adapted model receives approximately the same text-side signal as it would have received from the explicit style phrase.

4. Method: Text-Conditioning Style Absorption

4.1 Overview

The pipeline has six stages. No stage runs the DiT or the VAE on the local machine.

prompt pairing → official text-only encoding → alignment and Δ

→ gates, nulls, soft-prompt test → rank-1 init + SVD/LS absorption

→ packaged LoRA → hosted Turbo four-condition validation

4.2 Prompt-pair construction

A target style is admitted only if the base model already responds to it when the style phrase is written explicitly. Watercolor, oil painting, pencil sketch, and generic anime rendering are admissible starting points. A private studio look that the model cannot produce from text is out of scope; there is then no internal representation to extract.

Each experiment uses a bank of 20 to 40 content prompts spanning objects, scenes, and figures, held out from a disjoint evaluation bank. Every content prompt is paired with several style formulations to reduce token idiosyncrasy, for example watercolor / watercolor painting / traditional watercolor illustration / hand-painted watercolor. Insertion is prefix-style, not a free paraphrase of the whole sentence, so that the residual difference is dominated by the style span rather than by a rewritten scene.

4.3 Text-only encoding and alignment

Pairs are encoded with the official Qwen3-VL-4B checkpoint used by Krea 2, in text-only mode, on CPU. The vision tower is not loaded. Features are taken from the same twelve-layer tap that Krea 2 uses at inference. Substituting CLIP or a different Qwen checkpoint produces a direction that cannot be legally added to Krea's injection maps.

Style phrases change sequence length. A naive subtraction of two variable-length feature matrices mixes a genuine style shift with a token-alignment artifact. The encoder output is therefore split. Shared content tokens are aligned by a deterministic procedure such as longest common subsequence on the tokenized strings. The residual style span is

pooled separately. Pooling must be specified and held fixed; mean pooling is the default, token-weighted pooling an ablation. Because the official condition is a 12-layer stack rather than a single vector, implementations should store layer-wise features and not collapse the layer axis until the absorption stage forces a choice. Two statistics are stored for each pair i:

$$\Delta_i^{pool} = pool(E(P_{c+s}^{\,i})) - pool(E(P_c^{\,i}))$$
$$\Delta_i^{span} = pool(E(P_{c+s}^{\,i})[style\ tokens])$$

The mean style direction and mean content feature are

$$\mu = (1/N)\,\Sigma_i\,\Delta_i^{pool},\ \bar{h} = (1/N)\,\Sigma_i\,h_i$$

4.4 Style-direction testing and null controls

A single pair cannot establish that a difference is stylistic. Pairwise cosine similarities among the pooled differences are computed across contents. PCA is then run on the matrix whose rows are those differences. The redesigned protocol adds three null controls: (a) an unrelated adjective, (b) a visual attribute that is not a style renderer, and (c) paraphrases of the intended style. A useful style direction should be more consistent for the style paraphrases than for the unrelated controls. If the style and null conditions produce comparable directions, synthesis stops. This is a feature of the method: CPU time should be spent only on styles the model already owns.

Gate	Statistic	Proceed if
Directional consistency	Mean pairwise cosine of pooled Δ_i	Clearly above the empirical null floor. A working target is □ 0.4, treated as a provisional criterion, not a theoretical constant.
Rank concentration	$\lambda_1 / \Sigma_k \lambda_k$ from PCA	The leading component explains a disproportionate share of variance relative to later components.
Style specificity	Style paraphrases vs. null controls	Style paraphrases are more coherent than unrelated adjectives or non-renderer attributes.

Table 1. Cheap numerical gates that abort a style before any LoRA file is written.

4.5 Soft-prompt baseline

Before writing any LoRA, evaluate a direct embedding intervention. For held-out prompts, compare the native content feature h with h + αμ (or the corresponding token-level intervention). This baseline asks the simplest possible question: does the estimated direction itself move the conditioning representation toward the styled prompt? If it fails here, weight absorption should not be expected to rescue the method. The decomposition is diagnostic. If h + μ reproduces the desired shift but the LoRA does not, the failure lies in absorption or target-module choice. If even the soft prompt fails, the extracted direction is not a useful representation of the intended style.

4.6 Analytic rank-1 initializer

Let $W \in R^{d\_out \times d\_in}$ be one text-injection matrix, read from the RAW checkpoint by named-key lazy load. The default matrices are `txt_in.linear_1` / `txtmlp.1`, `txt_in.linear_2` / `txtmlp.3`, and `text_fusion.projector` / `txtfusion.projector`. The 28 DiT blocks are never opened.

Write h for a pooled or token-wise content feature and μ for the estimated style shift. The desired first-order behavior is that the styled projection applied to content features reproduce the base projection applied to styled features:

$$(W + \Delta W)\, h \approx W\,(h + \alpha\mu) = W\,h + \alpha\, W\mu$$

which is satisfied by any ΔW whose action on typical content features equals α Wμ.

Let $\bar{h}$ be the mean pooled content feature over the extraction set. The correct rank-1 construction, restored from the first draft, uses the unnormalized dual vector that enforces the identity constraint on the mean feature:

$$v = \bar{h} / \|\bar{h}\|_2^2 \text{ so that } v^T \bar{h} = 1$$

$$\Delta W_{R1} = \alpha\,(W\mu)\, v^T,\ B = \alpha\, W\mu,\ A = v^T$$

Do not replace the squared norm in the denominator by the unsquared norm. That substitution scales ΔW by $\|\bar{h}\|$.

When PCA retains r leading components $\mu_1, \ldots, \mu_r$, the same construction is applied to each component and the factors are concatenated, producing a rank-r adapter with r {1, 2, 4, 8}. Rank 1 is retained as a cheap, transparent initializer and as a first experiment. It is not treated as the primary optimal solution.

4.7 Primary method: thin-SVD / least-squares absorption

The rank-1 formula uses a single mean feature as a surrogate for the identity constraint. The primary synthesis method replaces that surrogate with an explicit low-rank fit on cached paired features. Let $H_c^{(i)}$ and $H_{c+s}^{(i)}$ be aligned feature matrices for pair i. For each target map W one solves

$$\min_{B,A} \Sigma_i \| (W + BA)\, H_c^{(i)} - W\, H_{c+s}^{(i)} \|_F^2$$

Equivalently, define the desired output residual $R_i = W(H_{c+s}^{(i)} - H_c^{(i)})$ after the chosen alignment. Stack the feature/output pairs and obtain a rank-r approximation of the implied residual map. A thin SVD provides a deterministic CPU solution for the unconstrained residual approximation; a few alternating least-squares sweeps can then be used if the exact LoRA factorization is required. Gradients never enter the encoder and never enter the DiT. Sequence-length mismatch is handled by the same content-token alignment used in Section 4.3; unaligned style tokens contribute only through their pooled increment.

Recommended ranks are r {1, 2, 4, 8}. Rank 1 tests the central hypothesis with minimal capacity. Rank 4 is the default for a first serious validation. Rank 8 is an escalation only when lower ranks produce a measurable but incomplete effect, typically a palette shift without a change in rendering tendency.

4.8 Target-module strategy

The smallest useful target set should be tested first. The recommended order is: the fusion projector, then the two text-width linears, then selected fusion-block linears only if the earlier targets show a clear style direction but insufficient visual expressivity. Attention projections and the bulk of the DiT remain untouched in the CPU method.

4.9 Packaging a loadable LoRA

The synthesizer must verify the exact key names in the selected Krea 2 release before serialization. Aliases in Appendix A are implementation checklists, not assumptions that a loader will silently correct. The synthesizer writes one namespace and, if needed, a second file with the aliased keys. Scaling follows the common convention lora_alpha = rank, so that a hosted slider of 1.0 corresponds to the constructed ΔW. Tensors are stored in bf16 or fp16. No dummy keys are written for untouched DiT blocks; loaders that ignore missing targets will apply a text-only adapter cleanly.

5. Experimental Protocol

5.1 Local compute budget

The intended local machine has a CPU, enough RAM to hold a quantized 4B text encoder (on the order of 4–8 GB for a 4-bit or 8-bit build), temporary space for feature caches, and disk access to the RAW safetensors file. The DiT weights other than the named text-injection matrices are not materialized. Image decoding is never performed locally. Generation and visual inspection run on a hosted endpoint such as Tensor.Art, Krea, or another Krea-compatible environment.

5.2 Phased experiment

Phase	Goal	Local work	Stop rule
0. Admissibility	Confirm explicit style response	None. Hosted generations of P_c versus P_{c+s}	If explicit prompting does not change style, abandon the target
1. Direction	Estimate a stable text-feature shift	Encode 20–40 pairs; cosine, PCA, nulls	Fail the gates in Table 1
1b. Soft prompt	Test the direction before packaging	Compare h and h+αμ on held-out prompts	If the embedding shift fails, do not write a LoRA
2. Absorption	Construct adapters	Rank-1 initializer + rank-4 SVD/LS on projector and txt_in	None; files are cheap
3. Validation	Test default-style prior	Upload .safetensors to a hosted Turbo endpoint	See Section 5.3
4. Escalation	Recover rendering, not only palette	Raise rank to 8; add fusion-block linears	If still palette-only, report the surrogate ceiling

Table 2. A CPU-first plan that spends local compute only after a style has cleared cheap gates.

5.3 Four-condition visual test

All images are generated on a hosted Krea 2 Turbo endpoint. Seeds, sampler, resolution, and platform settings are frozen across conditions. The evaluation prompt is drawn from the held-out bank and does not appear in extraction. The decisive comparison is not whether the proposed LoRA looks attractive in isolation; it is whether removing the explicit style word while enabling the adapter produces an output closer to the explicit-style baseline.

Condition	Prompt	Adapter	What it isolates
A. Baseline	a girl walking through a forest	off	The model’s default prior
B. Explicit style	a watercolor girl walking through a forest	off	The native style concept
C. Proposed LoRA	a girl walking through a forest	on	Default-style transfer without the style word
D. Combined	a watercolor girl walking through a forest	on	Interference or stacking with the native concept

Table 3. Four-condition protocol that separates default-style transfer from explicit prompting.

The decisive comparison is A → C. If C moves toward B in palette, edge character, and surface quality, the adapter has converted a conditional behavior into a prior. B → C asks how much of the native concept was recovered; identity is not required. D detects destructive interference. Strength is swept through {0.25, 0.50, 0.75, 1.00, 1.25}. A monotone response to the sweep is supporting evidence that the update is a direction rather than noise. Composition and object identity should remain stable.

5.4 Held-out generalization

Extraction concepts and evaluation concepts are disjoint. A style prior should transfer across semantic categories. If the effect is strong only on concepts used during extraction, the result is more plausibly a semantic or co-occurrence artifact than a reusable style prior.

Role	Concepts
Extraction	tiger, castle, woman, spaceship, fruit, motorcycle, jungle path, harbor
Evaluation	dog, portrait, train, landscape, robot, food, library interior, night market

Table 4. Content split used to test whether the adapter is a style prior or a concept-specific hack.

5.5 Quantitative companions to visual inspection

Hosted platforms rarely expose velocity tensors, so image-space proxies are used. A CLIP or SigLIP direction between A and B defines an empirical style axis; C is projected onto that axis [10]. The same encoder can report content preservation between A and C. Neither metric replaces looking at the pictures. They exist to make a negative result harder to argue away and a weak positive result harder to overclaim.

5.6 Baselines

(1) No adapter. (2) Explicit style prompt, no adapter. (3) Official image-trained style LoRAs released for Krea 2, such as the watercolor adapter in the public collection, when the target style has an official counterpart [9]. (4) A random rank-matched LoRA on the same target modules, to reject the hypothesis that any text-side perturbation looks like style. (5) The direct soft-prompt / mean-offset embedding intervention before LoRA packaging, which tests whether the closed-form packaging step itself loses information.

6. What Success Can and Cannot Mean

A successful C condition is not an aesthetically maximal watercolor. It is a measurable drift of the default prior toward the native style concept, on held-out contents, with a monotone strength sweep, obtained without images and without a GPU. That is a weaker object than a production style LoRA, and it is the object this method is allowed to claim.

6.1 Plausible positive behavior

- A shift in palette or tonal atmosphere.
- More consistent paper, pigment, or surface character.
- Systematic edge softness or mark-making tendency.
- A global rendering bias that persists across unrelated content.

6.2 Specific failure modes

- Tint-only. The adapter changes color statistics but not rendering character.
- Semantic leakage. The style direction also encodes objects or scene concepts associated with the phrase.
- Structural distortion. Pooled feature shifts alter composition, object identity, or geometry.
- Token-local effect. The direction disappears when the exact wording is paraphrased.
- RAW-to-Turbo mismatch. A text-side update derived from RAW does not transfer cleanly after distillation.
- Key mismatch. The serialized adapter loads but targets the wrong module and is effectively inert.
- Surrogate ceiling. Important style information resides in mid/late DiT blocks that the CPU method never edits.

These failure modes are expected, not exceptional. Official image-trained Krea 2 style LoRAs themselves often still require a trigger phrase to express a specific look rather than a generic illustrated prior. TCSA is a weaker intervention than those LoRAs. Comparing it against an official watercolor adapter and declaring defeat because the brushwork is weaker would miss the claim. Comparing it against the explicit style prompt and asking how much of that prompt can be retired is the correct test.

7. Discussion

The original study changed the role of the pretrained model from student to teacher. That inversion is worth keeping. What does not travel is the assumption that the teacher signal must be a velocity. On a CPU, the only teacher signal one can actually collect is the signal that lives in the language pathway. Packaging that signal as LoRA is a way of turning a conditional token into a prior: the user no longer has to write watercolor because the adapter permanently biases the maps that would have read that word.

This is closer to absorbing a soft prompt into weights than to training a style model. The strongest scientific hypothesis is therefore not that visual style is universally linear in language space. It is that some pretrained visual styles are sufficiently linear and content-independent in the model's conditioning space to be absorbed by a low-rank parameter update. That hypothesis is falsifiable with the gates, null controls, soft-prompt test, held-out concepts, and four-condition evaluation.

If later access to a GPU becomes available, the repaired velocity objective in Section 3 can be restored as a second stage, initialized from the text-side LoRA produced here. TCSA is then not a competitor to Concept Sliders; it is a feasible initializer and, for many users, the only complete method they can run.

8. Broader Use

Any named visual attribute that the official encoder represents stably is a candidate: cinematic photography, oil painting, pencil sketch, vintage photography, concept-art lighting, editorial illustration. Attributes that the model treats as objects rather than renderers — a specific living artist, a private product finish — are not. The same pipeline can in principle emit a negative adapter by flipping the sign of μ, which would suppress a style the model otherwise applies too freely. That experiment is optional and should not be conflated with the main feasibility claim.

9. Limitations

Known-style dependence. The method reorganizes knowledge already present in the pretrained model. It cannot create a visual style for which the base model has no usable representation.

Surrogate gap. Aligning text-injection maps is not the same as aligning velocity fields. High-frequency brushwork, local texture, or style information encoded deeper in the DiT cannot be written by editing `txt_in`. This is the price of refusing to run the backbone.

Approximate linearity. The construction assumes that a style phrase acts as an approximately additive and reusable shift. Styles that rewrite composition, camera, or object identity as a function of content will leak into μ and then into the LoRA.

Checkpoint fidelity. Exact Qwen3-VL aggregation, tokenizer behavior, the 12-layer tap, tensor shapes, and Krea 2 key mapping must match the checkpoint and inference implementation. An unofficial text encoder, a missed layer in the multi-layer stack, or a swapped linear_1 / linear_2 produces a file that loads and does nothing useful.

RAW-to-Turbo transfer. Official image LoRAs are trained so that RAW updates apply on Turbo. A closed-form text-side update is not covered by that training contract and must be tested. Successful transfer is an empirical question, not a mathematical consequence.

Hosted evaluation bias. External platforms may apply unpublished post-processing. The four-condition protocol should be run inside one platform with fixed settings, not compared across platforms.

10. Conclusion

Extracting a style LoRA from a pretrained text-to-image model without images is a reasonable research goal. Doing it by differencing the velocity field of a 12B flow-matching DiT is not a CPU method. This paper relocates the same goal to the text-conditioning pathway of Krea 2 and formalizes the resulting method as Text-Conditioning Style Absorption.

The workflow is staged on purpose: verify that the style is already expressible; measure paired text-feature differences on the official encoder; reject unstable directions with cosine, PCA, and null controls; test the direct soft-prompt intervention; construct a correct rank-1 initializer; refine it with low-rank least-squares / SVD absorption into `txt_in` and `text_fusion`; and evaluate the adapter on held-out prompts using a four-condition hosted Turbo test that asks whether the style word can be removed from the prompt.

The primary feasibility question is therefore no longer whether a pleasing picture can be made. It is whether a style that already lives in the model's language pathway can be isolated well enough to become a reusable, content-independent, low-rank parameter modification, on a machine that cannot run the image backbone at all. If that narrower demonstration succeeds, it is evidence that some forms of style adaptation are reorganization of knowledge the model already has — and that the cheapest place to reorganize that knowledge is the place where language first becomes conditioning.

References

[1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," ICLR, 2022.

[2] R. Gandikota, J. Materzynska, T. Zhou, A. Torralba, and D. Bau, "Concept Sliders: LoRA adaptors for precise control in diffusion models," 2023.

[3] Krea AI, "Krea 2," model card and technical documentation for RAW and Turbo checkpoints, 2026. https://github.com/krea-ai/krea-2

[4] Krea AI, "Krea 2 RAW," Hugging Face model card, 2026. https://huggingface.co/krea/Krea-2-Raw

[5] Hugging Face Diffusers, "DreamBooth LoRA training for Krea 2," README_krea2.md, 2026.

[6] kohya-ss, "Musubi Tuner: Krea 2 LoRA training notes," including default targets of 264 Linear layers, 2026.

[7] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," ICLR, 2023.

[8] Qwen Team, "Qwen3-VL," technical report and 4B instruct checkpoint used as Krea 2 text encoder.

[9] Comfy-Org, "Krea 2 pack and official style LoRA collection," Hugging Face, 2026.

[10] A. Radford et al., "Learning transferable visual models from natural language supervision," ICML, 2021.

[11] First draft: "Zero-shot style extraction and closed-form LoRA distillation from text-conditioning pathways: a CPU-feasible reformulation for Krea 2 RAW."

[12] Revised draft: "Text-conditioning style absorption for Krea 2 RAW: a CPU-feasible, dataset-free reformulation of zero-shot style LoRA construction."

[13] Original velocity-space feasibility draft: "Zero-shot style extraction and LoRA distillation from a pretrained text-to-image model: a feasibility study on extracting model-internal style representations from Krea 2 RAW."

Appendix A. Target Modules and Key Aliases

The synthesizer should prefer the smallest module set that can absorb a style prior. The recommended order of inclusion is projector, then the two txt_in linears, then fusion block linears only if Phase 4 is triggered. The mapping below must be verified against the actual checkpoint before any adapter is serialized.

Diffusers / PEFT key	Native / Musubi key	Role
txt_in.linear_1	txtmlp.1	First text-width projection
txt_in.linear_2	txtmlp.3	Second text-width projection
text_fusion.projector	txtfusion.projector	Layer-mixer into the DiT stream
text_fusion.layerwise_blocks.*	txtfusion.layerwise_blocks.*	Optional escalation
text_fusion.refiner_blocks.*	txtfusion.refiner_blocks.*	Optional escalation
transformer_blocks.*.attn.to_q / to_k / to_v / to_out.0	blocks.*.attn.wq / wk / wv / wo	Do not touch on CPU

Table A1. Key aliases. Only the first three rows are used in the default CPU method.

Appendix B. Worked Construction

B.1 Rank-1 initializer (from the first draft, algebra restored)

1. Encode each pair with the official text-only stack. Align content tokens. Store pooled content features h_i and pooled differences Δ_i.
2. Set $\mu = (1/N)\ \Sigma_i\ \Delta_i$ and $\bar{h} = (1/N)\ \Sigma_i\ h_i$. Stop if mean pairwise $\cos(\Delta_i, \Delta_j)$ is near the null floor, or if style paraphrases are no more coherent than null controls.
3. Confirm that $h + \alpha\mu$ moves held-out content features toward $E(P_{c+s})$. If this soft-prompt test fails, do not proceed.
4. For each target matrix W, compute $u = W\mu$ and $v = \bar{h} / \|\bar{h}\|_2^2$.
5. Write $B = \alpha u$ as a column and $A = v^T$ as a row. Default $\alpha = 1$.
6. Serialize A and B under the chosen namespace with lora_alpha = 1 for rank-1.

B.2 Primary SVD / least-squares absorption (from the revised draft)

1. Keep the same aligned feature caches $H_c^{(i)}$ and $H_{c+s}^{(i)}$.
2. For each W, form residuals $R_i = W(H_{c+s}^{(i)} - H_c^{(i)})$.
3. Stack features and residuals. Compute a rank-r thin SVD of the implied residual map, r {1, 2, 4, 8}.
4. Optionally run a few alternating least-squares sweeps to enforce the exact BA factorization used by the target loader.
5. Serialize with lora_alpha = r. Validate on Turbo with Table 3. Sweep strength. Do not iterate on extraction prompts that appear in the test set.

Appendix C. How the Two Drafts Were Merged

Table C1 records the mapping from the motivating velocity-space idea through the two text-pathway drafts into this integrated paper. Table C2 records the specific choices taken when the two text-pathway drafts disagreed.

Motivating velocity draft	This integrated paper
Teacher signal is a velocity difference D_s	Teacher signal is a text-feature difference Δ
Objective accidentally targeted D_s as a model output	Corrected objective: match styled conditioning from content input
PCA on velocity tensors across seeds and timesteps	PCA on encoder differences across contents only
LoRA trained by backprop through the 12B DiT	Rank-1 initializer plus thin SVD / LS; no DiT in the loop
Target-module search over attention, MLP, and fusion	Default targets restricted to txt_in and text_fusion
CPU pipeline by skipping VAE decode	CPU pipeline by never loading the DiT blocks
Claim framed as general zero-shot style creation	Claim framed as reorganization of an already-known style prior

Table C1. Mapping from the motivating velocity-space draft to the integrated method.

Point of disagreement	First draft	Revised TCSA draft	Integrated choice
Method name	Zero-shot closed-form distillation	Text-Conditioning Style Absorption (TCSA)	TCSA, with closed-form as the initializer
Primary synthesizer	Rank-1 formula as default proof of concept	Thin-SVD / LS as primary; rank-1 as initializer	Follow the revision: SVD/LS primary
Rank-1 vector v	$v = \bar{h} / \|\|\bar{h}\|\|^2$ (correct; $v^T\bar{h} = 1$)	$v = \bar{h} / \|\|\bar{h}\|\|$ (unit vector; scales ΔW by $\|\|\bar{h}\|\|$)	Restore the first draft's squared-norm formula
Null controls	Cosine and PCA only	Unrelated adjective, non-renderer attribute, paraphrases	Adopt the revision's three nulls
Soft-prompt test	Listed among baselines	Required step before writing LoRA	Required step (Phase 1b)
Architectural claim	Style first appears in the text pathway	Weaker claim: a style signal is observable there	Adopt the weaker, testable claim
Key-name status	Presented as usable aliases	Presented as hypotheses to verify	Checklist that must be verified on-disk
Feature geometry	Pooled encoder features	Pooled features; layer stack mentioned	Keep the 12-layer tap explicit; do not collapse early

Table C2. Reconciliation of the two source drafts. Where they conflict, the integrated paper states the chosen rule in the main text.

The original research question is unchanged. The feasible method is the union of the two drafts after those conflicts are resolved. If a later implementation shows that condition C consistently approaches condition B on held-out prompts, the narrower demonstration is enough: some visual styles in Krea 2 are already linear enough, in language space, to be packaged as a LoRA without ever generating a training image.